설비 고장을 예측하는 AI 데이터 분석

튜플과 셋

튜플로 묶기 → 셋으로 비교 → 센서 데이터 다루기

posco

CONTENTS

목차

01 튜플로 묶기

02 셋으로 비교

posco

posco

01

튜플로 묶기

짝지어진 데이터를 하나로 묶어 안전하게 관리한다

posco

짝지어진 데이터를 하나로 묶기

관련된 데이터는 따로 두지 말고 하나로 묶어 관리한다.

관련된 데이터는 따로 두지 말고
하나로 묶어서 관리한다.

따로 두면 순서가 어긋나 데이터 전체가 망가진다.

센서 이름과 값을 따로 두면 생기는 일

이름과 값이 따로 다니면 순서가 어긋나는 순간 짝이 깨진다.

01 · 짝 어긋남

이름·값을 따로 두었을 때

따로 두면 순서 틀어질 때
짝이 깨진다

값의 순서를 바꾸는 순간
모터온도에 베어링진동 값이 붙어
데이터 전체가 망가진다.

02 · 택배 비유

송장과 상자

송장과 물건이 따로 오면
짝을 못 맞춘다

짝으로 다니는 데이터는 처음부터 붙여둬야 안전하다.

03 · 오늘 배울 세 도구

묶기 · 거르기 · 찾기

묶는 튜플,
거르는 셋,
찾는 딕셔너리

오늘은 이 중 튜플과 셋을 다룬다.

튜플 만들기

소괄호 안에 값을 쉼표로 구분해 묶으면 튜플이 만들어진다.

01 · 정의

리스트와 차이

소괄호 ( ) 안에 쉼표로 구분

리스트가 대괄호를 썼다면 튜플은 소괄호.
문자열·숫자 섞어도 되고, 개수도 자유롭다.
좌표·연월일처럼 짝지어진 값을 묶는 발상이다.

02 · 단일값 함정

값이 하나뿐인 튜플

값 뒤에 쉼표 필수

1. (78) 은 그냥 숫자
2. (78,) 은 튜플

괄호만 치면 파이썬은 계산용 괄호로 해석한다.

튜플 만들기 — 코드로 확인

센서 이름과 값을 하나의 튜플로 묶는다.

CODE · 센서 이름과 값을 하나의 튜플로 묶기

```
sensor = ("모터온도", 78)
print(sensor)
one = (78,)
not_tuple = (78)
```

쉼표가 있는 one은 튜플, 쉼표 없는 not_tuple은 그냥 숫자 78이다.

튜플 인덱싱으로 값 꺼내기

대괄호 안에 순서 번호를 넣어 값을 꺼낸다.

01 · 0번부터 시작

시작점에서 몇 칸 떨어졌는지

첫 번째 값은 0번

컴퓨터는 시작점에서 몇 칸 떨어졌는지로 센다.
첫 번째 값은 0칸 떨어져 있으니 0번.
"영, 일, 이"로 세는 습관을 들이자.

02 · 뒤에서부터 세기

음수 인덱스

마지막 값은 -1번

음수를 넣으면 뒤에서부터 센다.
전체 길이를 몰라도 -1번이면
항상 맨 끝 값에 접근할 수 있다.

posco

튜플 인덱싱 — 코드로 확인

번호로 튜플의 값을 꺼낸다.

CODE · 번호로 튜플의 값을 꺼내기

```
sensor = ("모터온도", 78, "C")
print(sensor[0])
print(sensor[-1])
```

sensor[0] → 모터온도 / sensor[-1] → 전체 길이 몰라도 맨 끝 값 C

튜플 슬라이싱으로 구간 자르기

대괄호 안에 시작과 끝 번호를 콜론으로 적어 구간을 잘라낸다.

01 · 끝 번호 규칙

가장 헷갈리는 지점

끝 번호는 포함되지 않는다

1번부터 4번이라고 적으면 1, 2, 3번만 나온다.
잘라낸 개수가 (끝 – 시작)으로 정확히 맞아 계산이 깔끔하다.

02 · 생략 규칙

시작·끝은 생략 가능

빵을 저며 자르듯
한꺼번에 잘라낸다

시작을 생략하면 처음부터,
끝을 생략하면 끝까지 가져온다.

posco

튜플 슬라이싱 — 코드로 확인

시작과 끝 번호로 구간을 잘라낸다.

CODE · 시작과 끝 번호로 구간 잘라내기

```
readings = (78, 80, 79, 95, 77)

print(readings[1:4])
print(readings[:3])
print(readings[2:])
```

[1:4] → (80, 79, 95) · [:3] → 처음 세 개 · [2:] → 2번부터 끝까지

튜플 언패킹으로 한 번에 풀기

등호 왼쪽에 변수를 나열하면 튜플 값이 순서대로 담긴다.

01 · 패킹과 언패킹

짐 싸는 게 패킹, 푸는 게 언패킹

반복마다 이름과 값

자동 분리

등호 왼쪽에 변수를 쉼표로 나열하면
오른쪽 튜플 값들이 순서대로 변수에
쏙쏙 들어간다. 인덱스가 필요 없다.

02 · 의미가 드러나는 코드

개수가 정확히 같아야 함

인덱스 없이 의미가

드러나는 코드

변수 이름이 그 값이 무엇인지
설명해 주어 읽기에도 좋다.
변수 개수와 값 개수는 정확히 일치해야 한다.

튜플 언패킹 — 코드로 확인

인덱스 없이 한 번에 변수로 분리한다.

CODE · 인덱스 없이 한 번에 변수로 분리

```
sensor = ("모터온도", 78)

name = sensor[0]
value = sensor[1]
name, value = sensor
```

맨 아래 한 줄이 언패킹 — name에 "모터온도", value에 78이 자동으로 담긴다.

posco

튜플의 불변성 이해하기

튜플은 한 번 만들면 값을 바꾸거나 추가·삭제할 수 없다.

01 · 리스트

연필 글씨

연필 글씨처럼

수정 가능

리스트는 만든 뒤에도 값을
자유롭게 바꿀 수 있다.
지우고 다시 쓸 수 있다.

02 · 튜플

볼펜 글씨

볼펜 글씨처럼

고정

바꿀 수 없는 성질을 불변이라 한다.
볼펜으로 쓴 영수증이 오히려
믿을 수 있는 기록이 되듯,
튜플의 불변성은 신뢰의 원천이다.

posco

튜플의 불변성 — 코드로 확인

값을 바꾸려 하면 오류가 발생한다.

CODE · 값을 바꾸려 하면 오류 발생

```
sensor = ("모터온도", 78)

sensor[1] = 80
# TypeError 발생
# 튜플은 값을 바꿀 수 없음
```

posco

왜 바꿀 수 없는 튜플을 쓰는가

바꿀 수 없다는 점이 곧 실수로 망가지지 않는 안전장치다.

01 · 안전장치

바꿀 수 없다는 점이 곧 실수로 망가지지 않는 안전장치

02 · 즉시 차단

실수로 건드리면 그 자리에서 바로 오류로 막아준다

03 · 키 사용

바뀌지 않아 딕셔너리의 키로도 사용 가능

정리하면 튜플의 "못 바꿈"은 제약이 아니라 **보호**다.

튜플과 리스트의 차이

바뀔 데이터는 리스트, 고정 데이터는 튜플로 선택한다.

구분	리스트	튜플
괄호	대괄호 [ ]	소괄호 ( )
값 변경	가능	불가능
무게	보통	가벼움
적합	바뀔 값	고정 값

실무에선 센서 하나를 튜플로 묶고, 그 튜플들을 리스트에 담아 쓴다.

posco

튜플 다루기 — len·count·index

튜플을 다룰 때 자주 쓰는 세 가지 도구다.

01 · LEN

`len(t)`

전체 값이 몇 개인지 개수 세기

길이, 즉 전체 개수를 센다.
데이터가 비지 않았는지 확인할 때 쓴다.

02 · COUNT

`t.count(value)`

특정 값이 몇 번 나오는지 세기

경고가 몇 번 울렸는지 셀 때 유용하다.

03 · INDEX

`t.index(value)`

특정 값의 첫 위치 번호 찾기

처음 나오는 위치를 알려준다.
단, 찾는 값이 없으면 오류가 나니 주의.

posco

len·count·index — 코드로 확인

개수, 빈도, 위치를 한 번에 확인한다.

CODE · 개수, 빈도, 위치를 한 번에 확인

```
status = ("정상", "정상", "경고", "정상")

print(len(status))
print(status.count("경고"))
print(status.index("경고"))
```

len → 4 (전체) · count → 1 (경고 한 번) · index → 2 (경고는 2번 자리)

여러 센서를 튜플 리스트로 모으기

센서 정보 튜플을 리스트에 담아 여러 센서를 한 번에 관리한다.

01 · 튜플 안의 짝 보호

바깥은 리스트, 안은 튜플

각 센서는 튜플로
짝 보호

각 센서의 이름과 값은
튜플로 단단히 묶여 짝이 어긋나지 않는다.

02 · 리스트의 자유로움

큰 상자 안의 작은 상자

전체는 리스트로
자유롭게 추가

큰 리스트 상자 안에
작은 튜플 상자들이 줄지어 있는 모습.
오늘 가장 많이 쓰는 형태다.

posco

튜플 리스트 — 코드로 확인

튜플들을 리스트에 모은 구조다.

CODE · 튜플들을 리스트에 모은 구조

```
sensors = [
    ("모터온도", 78),
    ("베어링진동", 0.5),
]

print(len(sensors))
```

대괄호로 감싼 리스트 안에 소괄호로 묶은 튜플들이 들어 있다.

posco

튜플 리스트를 반복문으로 처리하기

반복문과 언패킹을 함께 쓰면 모든 센서를 한 번에 처리한다.

```
for name, value in sensors
```

오늘의 하이라이트 — 반복마다 이름과 값이 자동으로 분리된다

01 · 자동 분리

반복마다 이름과 값을 자동 분리

튜플 리스트를 for에 넣고 변수 두 개로 받으면 언패킹까지 한 번에 된다.
인덱스가 필요 없다.

02 · 확장성

센서가 3개든 300개든 코드는 동일

데이터 개수가 늘어나도 코드는 그대로 — 이게 반복문의 힘이다.

posco

반복문 처리 — 코드로 확인

반복하며 기준 초과 센서를 검사한다.

CODE · 반복하며 기준 초과 센서 검사

```
sensors = [("모터온도", 78), ("펌프압력", 95)]
for name, value in sensors:
    if value > 90:
        print(name, "경고")
```

90을 넘는 펌프압력만 경고 출력 — 반복문 + 언패킹 + 조건문의 합작

튜플 안의 튜플로 메타데이터 담기

튜플 안에 또 튜플을 넣어 위치 좌표 같은 부가 정보를 함께 묶는다.

01 · 서랍 속의 서랍

큰 상자 안의 작은 상자

큰 상자 안의
작은 상자를 여는 것

위치는 가로세로 두 값이 한 쌍이니
작은 튜플로 만들어 큰 센서 튜플 안에 넣는다.

02 · 인덱스 두 번

바깥 자리 먼저, 안쪽 다음

인덱스를 두 번 써서
안쪽 값에 접근

여러 정보를 함께 담는 것을
"메타데이터를 담는다"고 표현한다.

posco

중첩 튜플 — 코드로 확인

인덱스를 두 번 써서 꺼낸다.

CODE · 인덱스를 두 번 써서 꺼내기

```
sensor = ("모터온도", 78, (3, 5))
print(sensor[2][0])
```

sensor[2] → (3, 5) · 거기에 [0] → 가로 좌표 3

sorted로 튜플 리스트 정렬하기

sorted는 튜플 리스트를 자동으로 정렬한다(첫 번째 값 기준).

01 · 정렬 기준

어느 값으로 줄 세울지

정렬 기준을 맨 앞에

정렬하고 싶은 값을
튜플의 맨 앞에 두면
sorted가 그 값 기준으로 정렬한다.

02 · 방향 전환

이상 신호를 맨 위로

reverse로 큰 값부터 정렬

이상 신호가 강한 센서를
맨 위로 올릴 때 유용하다.

sorted — 코드로 확인

측정값이 큰 순서로 정렬한다.

CODE · 측정값이 큰 순서로 정렬

```
sensors = [(78, "모터온도"), (95, "베어링진동"), (32, "펌프압력")]

hot = sorted(sensors, reverse=True)   # 값이 앞이라 값 기준 정렬
print(hot[0])   # (95, "베어링진동") 가장 높은 센서
```

값을 맨 앞에 두면 sorted가 값 기준으로 정렬 · reverse=True = 큰 값부터 · hot[0]은 가장 높은 센서

posco

튜플 활용 전체 흐름

묶기 → 모음 → 반복 처리 → 번호·정렬의 4단계 흐름이다.

STEP 1

튜플 묶기

짝지어진 데이터(이름·값)를 튜플로 묶어 짝이 어긋나지 않도록 보호한다.

```
(name, value)
```

STEP 2

리스트로 모음

그 튜플들을 리스트에 모아 여러 센서를 한 번에 관리한다.

```
[(...), (...), ...]
```

STEP 3

반복 처리

반복문과 언패킹으로 전체를 한 번에 처리한다.

```
for n, v in list
```

STEP 4

정렬

정렬이 필요하면 sorted를 더한다.

```
sorted
```

이 흐름이 센서 데이터를 다루는 기본 골격이다.

튜플 활용 정리 — 핵심

핵심 흐름은 튜플로 묶고 리스트로 모아 반복문으로 처리하는 것이다.

01 · 기본 골격

핵심 흐름은 튜플로 묶고 리스트로 모아 반복문으로 처리

이 흐름이 센서 데이터를 다루는 기본 골격이다.

02 · 판단 기준

안 바뀔 데이터는 튜플, 바뀔 데이터는 리스트

한 번 정해지면 안 바뀔 데이터는 튜플, 계속 추가·수정할 데이터는 리스트.

실습 1. 센서를 튜플로 묶고 꺼내기

센서 이름과 값을 튜플로 묶고 인덱싱·언패킹으로 꺼내기

목표

센서 이름과 측정값을 하나의 튜플로 묶고, 인덱싱과 언패킹으로 값을 꺼내기

단계

① 센서 이름과 측정값을 소괄호와 쉼표로 튜플에 저장
② 인덱스 0·1로 이름과 값을 각각 꺼내 출력
③ 언패킹으로 두 변수에 한 번에 받아 출력

예상 결과

('모터온도', 78)
모터온도
78
모터온도 78

posco

실습 1. 센서를 튜플로 묶고 꺼내기 정답

튜플로 묶고 인덱싱·언패킹으로 꺼내기

CODE · 정답

```
s1 = ("모터온도", 78)
print(s1)           # ('모터온도', 78)
print(s1[0])        # 모터온도
print(s1[1])        # 78
name, value = s1    # 언패킹
print(name, value)  # 모터온도 78
```

posco

실습 2. 튜플 리스트를 반복 처리하기

튜플 리스트를 for로 순회하며 출력하고 기준 초과를 경고

목표

(이름, 값) 튜플의 리스트를 for로 순회하며 값을 출력하고, 기준을 넘는 센서를 경고

단계

① 여러 센서를 (이름, 값) 튜플의 리스트로 저장
② for 언패킹으로 이름과 값을 하나씩 꺼내 출력
③ if로 값이 기준(90)을 넘으면 경고 출력

예상 결과

각 센서 이름·값 출력
회전속도 경고
펌프압력 경고

posco

실습 2. 튜플 리스트를 반복 처리하기 정답

for 언패킹 순회와 기준 초과 경고

CODE · 정답

```
sensors = [
    ("모터온도", 78),
    ("회전속도", 1750),
    ("펌프압력", 95),
    ("유량", 42),
]
for name, value in sensors:
    print(name, value)
limit = 90
for name, value in sensors:
    if value > limit:
        print(name, "경고")   # 회전속도 경고 / 펌프압력 경고
```

posco

실습 3. 중첩 튜플로 센서 위치 관리하기

중첩 튜플로 위치를 담고 언패킹·조건으로 골라내기

목표

(이름, 값, (x, y)) 중첩 튜플로 위치를 담고, 언패킹과 조건으로 특정 구역 센서를 골라내기

단계

① 각 센서를 (이름, 값, (x, y)) 중첩 튜플의 리스트로 저장
② for 언패킹으로 위치 튜플을 다시 x, y로 분리해 출력
③ if로 x가 기준(5) 이하인 센서만 골라 출력

예상 결과

각 센서 이름·위치 출력
x≤5 센서만: 모터온도, 펌프압력

posco

실습 3. 중첩 튜플로 센서 위치 관리하기 정답

중첩 튜플 언패킹과 조건 선별

CODE · 정답

```
sensors = [
    ("모터온도", 78, (3, 5)),
    ("베어링진동", 0.5, (7, 2)),
    ("펌프압력", 95, (4, 8)),
]
for name, value, pos in sensors:
    x, y = pos
    print(name, "위치:", x, y)
for name, value, pos in sensors:
    x, y = pos
    if x <= 5:
        print(name, "1구역")   # 모터온도 / 펌프압력
```

posco

02

셋으로 비교

중복을 자동으로 거르고, 두 목록의 차이를 한눈에 본다

중복된 센서 기록 정리하기

같은 값이 여러 번 들어온 데이터는 중복을 걸러야 정확한 분석이 가능하다.

중복을 걸러야
정확한 분석이 가능하다

셋은 같은 값을 두 번 담아도 자동으로 하나만 유지한다.

중복이 왜 문제가 되는가

중복을 그대로 두면 결과가 왜곡되고, 셋이 한 줄로 해결한다.

01 · 중복 발생

기록 수집 과정

동시 수집·재전송으로 기록 중복

여러 장비가 같은 센서를 읽거나 데이터를 합치는 과정,
재전송 등에서 흔히 발생한다.

02 · 결과 왜곡

3종류가 10건으로

3종류가 10건으로 세어져 오판

설비 상태를 실제보다 나쁘게 판단하게 된다.

03 · 셋의 해결

한 줄로 끝

set으로 감싸면 중복 제거

리스트를 set으로 감싸기만 하면 중복이 사라진다.
게다가 두 목록 비교도 가능하다.

셋 만들기

중괄호로 만들며 중복된 값은 자동으로 하나만 남는다.

01 · 만드는 법

수학의 집합에서 온 개념

리스트를 set으로 감싸면 중복 제거

중괄호 { } 안에 값을 쉼표로 넣어도 되고,
리스트를 set( )으로 감싸도 된다.
중복이 없고 순서도 없다.

02 · 빈 셋 주의

빈 중괄호는 딕셔너리

빈 셋은 set( ) 함수로 생성

빈 중괄호 { }는 딕셔너리이므로
빈 셋은 반드시 set( ) 함수로 만든다.

posco

셋 만들기 — 코드로 확인

중괄호와 set으로 중복을 제거한다.

CODE · 중괄호와 set으로 중복 제거

```
ids = {"S01", "S02", "S01"}
print(ids)

logs = ["S01", "S02", "S01", "S03"]
unique = set(logs)
print(unique)
```

set( )으로 감싸는 한 줄이 중복 제거의 가장 빠른 방법

셋의 두 가지 특징

순서가 없어 인덱스로 값을 꺼낼 수 없다.

01 · 무순서

순서가 없어 인덱스로 값을 꺼낼 수 없다

리스트·튜플은 0번 1번 순서가 있지만 셋은 그 개념 자체가 없다. 출력하면 넣은 순서와 다를 수 있다.

02 · 무중복

중복이 없어 같은 값은 항상 하나만 유지

"있다·없다"만 따지지 몇 개인지는 따지지 않는다. 종류 세기·포함 확인·목록 비교에 안성맞춤.

posco

add로 원소 추가하기

add는 셋에 새 값을 추가하며 이미 있는 값은 무시한다.

01 · append와 차이

리스트의 append와 비슷하지만

중복이 안 쌓인다

append는 같은 값을 또 쌓지만
add는 이미 있는 값을 그냥 무시한다.

02 · 중복 걱정 없음

오류도 안 난다

검사 없이
계속 추가 가능

경고 센서를 모을 때 같은 센서가
여러 번 울려도 그냥 add만 하면 된다.

posco

add — 코드로 확인

새 값 추가와 중복 무시를 확인한다.

CODE · 새 값 추가와 중복 무시

```
alerts = {"S01", "S02"}

alerts.add("S03")
print(alerts)

alerts.add("S01")
print(alerts)
```

이미 있는 S01을 다시 add해도 변화 없음 — 실시간 경고 모을 때 마음 놓고 add만

posco

in으로 포함 여부 확인하기

in 연산자로 특정 값이 셋에 있는지 빠르게 확인한다.

01 · True / False

값 in 셋 형태

있으면 True,
없으면 False

조건문과 함께 쓰면 특정 센서가
경고 목록에 있을 때만 알림을 보낼 수 있다.

02 · 빠른 검색

셋의 진짜 강점

데이터 많을수록
리스트보다 유리

리스트는 처음부터 하나씩 비교하지만
셋은 값의 자리를 미리 계산해 두어
거의 즉시 찾아낸다.

in — 코드로 확인

포함 여부 확인 후 처리한다.

CODE · 포함 여부 확인 후 처리

```
alerts = {"S01", "S02", "S03"}

print("S02" in alerts)
print("S09" in alerts)

if "S02" in alerts:
    print("S02 정비 알림")
```

S02 → True · S09 → False · 조건문과 결합하면 알림 처리도 가능

posco

union으로 합집합 구하기

union은 두 셋을 합쳐 중복 없는 전체 목록을 만든다.

01 · A ∪ B

동그라미 두 개의 전체 영역

겹치는 값은

한 번만 포함

A라인과 B라인의 센서를 union하면
두 라인의 모든 센서가 중복 없이 나온다.
공장 전체 센서 종류를 파악할 때 유용.

02 · 같은 일을 하는 두 표현

파이프 기호로도 가능

```
a.union(b)
```

```
a | b
```

두 표현 모두 같은 결과를 돌려준다.
라인이 여러 개면 차례로 union해 가면 된다.

posco

union — 코드로 확인

두 라인 센서를 합쳐 전체 종류를 파악한다.

CODE · 두 라인 센서를 합치기

```
line_a = {"S01", "S02", "S03"}
line_b = {"S03", "S04"}

all_s = line_a.union(line_b)
print(all_s)
```

결과: S01~S04 모두 등장 · 양쪽에 있던 S03은 한 번만

posco

intersection으로 교집합 구하기

intersection은 두 셋에 공통으로 든 값만 골라낸다.

01 · A ∩ B

동그라미 두 개의 겹치는 부분

양쪽 모두에 있는 것만

남긴다

합집합이 가장 넓다면 교집합은 가장 좁다.
두 라인 공통 센서로 부품 공유,
이틀 연속 경고 센서 추적에 강력하다.

02 · 두 표현

앰퍼샌드 기호로도 가능

```
a.intersection(b)
```

```
a & b
```

두 조건을 동시에 만족하는
것을 찾을 때 강력하다.

intersection — 코드로 확인

두 라인의 공통 센서를 찾는다.

CODE · 두 라인 공통 센서 찾기

```
line_a = {"S01", "S02", "S03"}
line_b = {"S03", "S04"}

common = line_a.intersection(line_b)
print(common)
```

양쪽에 있는 S03만 남음 · S01·S02는 A에만, S04는 B에만 있으니 빠짐

difference로 차집합 구하기

difference는 한 셋에는 있고 다른 셋에는 없는 값을 골라낸다.

01 · A − B ≠ B − A

A 동그라미에서 겹치는 부분 제거

순서에 따라

결과가 달라진다

A에서 B를 뺀 것과
B에서 A를 뺀 것은 다르다.
초승달처럼 한쪽만 남기는 그림.

02 · 변화 추적에 강함

두 시점 비교

`a.difference(b)`

`a - b`

오늘 경고에서 어제 경고를 빼면
오늘 새로 생긴 이상을 알 수 있다.
빼기 기호로도 가능.

difference — 코드로 확인

빼는 방향에 따라 다른 결과를 확인한다.

CODE · 빼는 방향에 따라 다른 결과

```
line_a = {"S01", "S02", "S03"}
line_b = {"S03", "S04"}

print(line_a.difference(line_b))
print(line_b.difference(line_a))
```

A − B → {S01, S02} · B − A → {S04} — 빼는 방향에 따라 완전히 다른 결과

집합 연산 세 가지 정리

동그라미 두 개가 겹친 그림 하나에 세 연산이 모두 들어 있다.

연산	의미	용도
합집합	합친 전체	종류 파악
교집합	공통인 것	공통점 찾기
차집합	한쪽만	변화 추적

전체가 궁금하면 **합집합**, 공통이 궁금하면 **교집합**, 차이가 궁금하면 **차집합**.

집합 연산 활용 — 핵심

두 목록 비교는 합집합·교집합·차집합 세 세트로 완전 분석한다.

01 · 완전 분석

두 목록 비교는 세 세트로 완전 분석

두 목록을 받으면 습관적으로 세 연산을 모두 구해보면 전체 규모·공통·각자 고유가 한눈에 정리된다.

02 · 한 줄 요약

전체는 합집합, 공통은 교집합, 차이는 차집합

두 라인을 비교할 때 양방향 차집합으로 각 라인 고유까지 알면 두 라인 관계가 완전히 그려진다.

이 집합 연산은 나중에 Pandas로 두 표를 비교할 때도 그대로 쓰인다.

posco

실습 4. 셋으로 중복 센서 제거하기

셋으로 중복 기록을 제거하고 종류 수 확인

목표

중복이 있는 센서 ID 기록을 셋으로 바꿔 중복을 제거하고, 정렬 출력·개수 확인

단계

① 중복이 있는 센서 ID 리스트를 저장
② set으로 변환해 중복을 제거
③ sorted로 정렬 출력하고 len으로 종류 수 확인

예상 결과

['S01', 'S02', 'S03']
종류 수: 3

posco

실습 4. 셋으로 중복 센서 제거하기 정답

set으로 중복 제거·정렬·개수

CODE · 정답

```
logs = ["S01", "S02", "S01", "S03", "S02"]
unique = set(logs)
print(sorted(unique))        # ['S01', 'S02', 'S03']
print("종류 수:", len(unique))  # 종류 수: 3
```

posco

실습 5. 두 라인의 센서 구성 비교하기

두 라인 셋으로 합집합·교집합·차집합 구하기

목표

두 라인의 센서 셋으로 전체(합집합)·공통(교집합)·차이(차집합)를 구하기

단계

① 두 라인의 센서 ID를 각각 셋으로 저장
② union으로 전체, intersection으로 공통 구하기
③ difference로 각 라인에만 있는 센서를 양방향으로 구하기

예상 결과

공통: {'S03', 'S05'}
A만: {'S01', 'S02'}
B만: {'S04'}

실습 5. 두 라인의 센서 구성 비교하기 정답

union·intersection·difference

CODE · 정답

```
line_a = {"S01", "S02", "S03", "S05"}
line_b = {"S03", "S04", "S05"}
print(line_a.union(line_b))         # 전체
print(line_a.intersection(line_b))  # {'S03', 'S05'}
print(line_a.difference(line_b))    # {'S01', 'S02'}
print(line_b.difference(line_a))    # {'S04'}
```

posco

실습 6. 두 시점의 이벤트 센서 추적하기

두 시점 셋으로 신규·지속 이상을 추적

목표

어제와 오늘의 이상 센서 셋으로 새로 생긴(신규) 이상과 계속되는(지속) 이상을 찾기

단계

① 어제와 오늘의 이상 센서 ID를 각각 셋으로 저장
② difference로 오늘만 있는 신규 이상 구하기
③ intersection으로 두 날 모두 있는 지속 이상 구하기

예상 결과

신규 이상: {'S05'}
지속 이상: {'S02', 'S03'}

posco

실습 6. 두 시점의 이벤트 센서 추적하기 정답

difference(신규)·intersection(지속)

CODE · 정답

```
yesterday = {"S01", "S02", "S03"}
today = {"S02", "S03", "S05"}
print(today.difference(yesterday))    # 신규: {'S05'}
print(today.intersection(yesterday))  # 지속: {'S02', 'S03'}
```